

# **Fiche technique détaillé**

The logo consists of a large, stylized, light red 'M' shape in the background. Overlaid on this is the text 'MediaStock' in a bold, black, sans-serif font.

**MediaStock**

## Table des matières

|                                                                    |    |
|--------------------------------------------------------------------|----|
| 1. Contexte, périmètre et objectifs.....                           | 4  |
| 2. Architecture logicielle.....                                    | 4  |
| 2.1 Vue d'ensemble.....                                            | 4  |
| 2.2 Arborescence du dépôt (principaux dossiers/fichiers).....      | 4  |
| 3. Conteneurisation du projet.....                                 | 5  |
| 3.1 Conteneurs utilisés.....                                       | 5  |
| 3.2 Volumes et persistance.....                                    | 6  |
| 3.3 Justification de l'utilisation de Docker.....                  | 6  |
| 3.4 Environnement de production.....                               | 7  |
| 4. Backend (PHP + PDO).....                                        | 7  |
| 4.1 Connexion et configuration de la base de données.....          | 7  |
| 4.2 Organisation du code et accès aux données.....                 | 9  |
| 4.3 Justification et bonnes pratiques backend.....                 | 10 |
| 5. Base de données.....                                            | 10 |
| 5.1 Schéma relationnel général.....                                | 10 |
| 5.2 Principales relations et contraintes.....                      | 11 |
| 5.3 Décisions de conception notables.....                          | 12 |
| 5.4 Choix techniques et justification.....                         | 12 |
| 5.5 Sécurité et intégrité des données.....                         | 12 |
| 5.6 Justification d'ensemble.....                                  | 13 |
| 6. Frontend.....                                                   | 13 |
| 6.1 Technologies utilisées.....                                    | 13 |
| 6.2 Structure et principales pages.....                            | 13 |
| 6.3 Comportements interactifs et ergonomie.....                    | 14 |
| 6.4 Justification des choix.....                                   | 14 |
| 7. QR Codes.....                                                   | 15 |
| 7.1 Rôle et fonctionnement.....                                    | 15 |
| 7.2 Génération des QR Codes.....                                   | 15 |
| 7.3 Lecture et utilisation.....                                    | 15 |
| 7.4 Convention d'identification.....                               | 16 |
| 7.5 Justification et bénéfices.....                                | 16 |
| 8. Sécurité, conformité et sessions.....                           | 16 |
| 8.1 Sécurisation des communications.....                           | 16 |
| 8.2 Authentification et gestion de sessions.....                   | 17 |
| 8.3 Protection contre les attaques web.....                        | 18 |
| 8.4 Sécurité des environnements Docker.....                        | 18 |
| 8.5 Conformité RGPD.....                                           | 19 |
| 8.6 Justification globale.....                                     | 19 |
| 9. Déploiement et exploitation.....                                | 20 |
| 9.1 Procédure d'installation (environnement de développement)..... | 20 |
| 9.2 Déploiement en production.....                                 | 20 |
| 9.3 Justification des choix d'exploitation.....                    | 21 |
| 9.4 Résumé.....                                                    | 21 |
| 10. Bonnes pratiques de développement.....                         | 22 |
| 10.1 Gestion du versionnement (Git).....                           | 22 |
| 10.2 Gestion de l'environnement Docker.....                        | 22 |
| 10.3 Tests et validation .....                                     | 23 |

|                                                                                    |    |
|------------------------------------------------------------------------------------|----|
| 10.4 Documentation et maintenance.....                                             | 23 |
| 11. Points d'attention et décisions techniques prises en cours de réalisation..... | 24 |
| 11.1 Configuration d'Apache et sécurité du DocumentRoot.....                       | 24 |
| 11.2 Organisation du backend : API centralisée.....                                | 24 |
| 11.3 Base de données et intégrité applicative.....                                 | 24 |
| 11.4 Hébergement et contraintes techniques.....                                    | 25 |
| 11.5 Synchronisation et dépendances Docker.....                                    | 25 |
| 11.6 Synthèse des décisions techniques.....                                        | 26 |
| 11.7 Bilan global.....                                                             | 26 |
| 12. Annexes utiles pour la reprise du projet.....                                  | 26 |
| 12.1 Cahier des charges détaillé.....                                              | 26 |
| 12.2 Fiche de réalisation.....                                                     | 26 |
| 12.3 Utilité de ces annexes pour la reprise.....                                   | 27 |
| 13. "Pourquoi ces choix ?" — Synthèse des décisions techniques.....                | 27 |
| 13.1 Docker : cohérence et portabilité.....                                        | 27 |
| 13.2 PHP + MySQL : simplicité, fiabilité et compatibilité.....                     | 27 |
| 13.3 DocumentRoot = public/ et .htaccess : sécurité et routage maîtrisés.....      | 28 |
| 13.4 API sous public/api : séparation claire front / back.....                     | 28 |
| 13.5 QR = ID : efficacité du flux de prêt / restitution.....                       | 28 |
| 13.6 Archivage logique : conformité et traçabilité.....                            | 29 |
| 14. Guide "Première contribution" (Check-list – 15 minutes).....                   | 29 |
| 14.1 Objectif.....                                                                 | 29 |
| 14.2 Étapes d'installation rapide.....                                             | 29 |
| 14.3 Découverte du code.....                                                       | 29 |
| 14.4 Contribution via Git.....                                                     | 30 |
| 14.5 Bonnes pratiques à respecter.....                                             | 30 |
| 15. Retour d'expérience des développeurs.....                                      | 30 |
| 15.1 Difficultés rencontrées.....                                                  | 30 |
| 15.2 Enseignements et axes d'amélioration.....                                     | 32 |
| 15.3 Bilan humain et technique.....                                                | 32 |
| 15.4 Évolutions prévues et perspectives d'évolution.....                           | 32 |

# 1. Contexte, périmètre et objectifs

*MédiaStock* est une application web “mobile-first” pour gérer l’inventaire et les prêts/restitutions de matériel (création d’items, génération/scan de QR codes, prêts/retours avec dates et notes d’état). Les choix techniques et organisationnels ont été définis pour remplacer un processus papier, garantir la traçabilité et accélérer les opérations sur le terrain.

Objectifs fonctionnels :

- **MUST** : CRUD inventaire, création/retour de prêts par QR code, authentification admin, unicité des items.
- **SHOULD** : échéances/retards, génération/impression de QR, modification d’un matériel.
- **COULD** : inventaire par scan, statistiques, multi-comptes/journal d’audit.

## Justification

Prioriser un socle opérationnel minimal (MUST) permet un déploiement rapide, tout en cadrant des évolutions désirables (SHOULD/COULD) sans bloquer la mise en service.

## 2. Architecture logicielle

### 2.1 Vue d’ensemble

Architecture en trois-tiers :

- **Présentation** : HTML/CSS/JS + Bootstrap, interface “mobile-first”.
- **Application** : PHP (procédural) en MVC léger.
- **Base de données** : MySQL via PDO.

## Justification

L’architecture trois-tiers sépare responsabilités et facilite la maintenance/l’évolutivité; le couple PHP/MySQL est adapté au contexte (volumétrie modeste, outillage standard, compétences disponibles).

### 2.2 Arborescence du dépôt (principaux dossiers/fichiers)

```
mediastock/
├── docker-compose.yml          # Configuration Docker (web, mysql, phpMyAdmin)
├── docker-compose.production.yml # configuration pour déploiement (ports restreints, sans
phpMyAdmin)
├── Dockerfile                  # image PHP/Apache avec mod_rewrite
├── .env.example                # variables d’environnement sans secrets
├── .env                        # variables sensibles (non versionné)
├── .gitignore                  # exclusion des fichiers sensibles et temporaires
├── public/                     # dossier exposé par Apache
│   └── api/                    # endpoints backend exposés au front
```

|                           |                                                       |
|---------------------------|-------------------------------------------------------|
| tests/                    | # fichier tests API/ intégration                      |
| .htaccess                 | # réécriture d'URL et accès frontend                  |
| index.php,                | # point d'entrée avec redirection                     |
| login.php,                | # authentification administrateur                     |
| login-verify.php,         | # vérification + timeout session                      |
| frontend/                 |                                                       |
| accueil.html,             | # page statique                                       |
| *.php,                    | # pages dynamiques                                    |
| *.css,                    | # feuilles de style                                   |
| *.js,                     | # scripts js (QR code, prêt, retour...)               |
| src/modèles               | # classes métiers (Item, Pret, Emprunteur, Formation) |
| config/                   |                                                       |
| database.php,             | # connexion PDO centralisée                           |
| env-example.php / env.php | # variables d'environnement pour Database.php         |
| sql/init.sql              | # script SQL (création et données initiales)          |

Le dossier *public/* constitue le *document root* d'Apache, afin de ne pas exposer le code applicatif ni le dossier *src/*.

Le dossier *public/api/* regroupe les *endpoints* consommés par le front-end via des requêtes *fetch*.

Nous avons choisi d'isoler le dossier *frontend/public* pour les raisons suivantes :

- **Sécurité** : seules les ressources statiques (assets) et le point d'entrée HTTP sont exposés publiquement.
- **Simplicité de routage** : le fichier *.htaccess* redirige automatiquement les URLs applicatives vers *index.php*.
- **Alignement MVC** : les vues, scripts JavaScript et feuilles de style sont placés côté *public/*, tandis que la logique applicative et l'accès aux données restent en dehors du *docroot*.

## 3. Conteneurisation du projet

### 3.1 Conteneurs utilisés

| Service           | Rôle                                                              | Port exposé       | Notes                                                                                                        |
|-------------------|-------------------------------------------------------------------|-------------------|--------------------------------------------------------------------------------------------------------------|
| <b>web</b>        | Serveur Apache + PHP 8.2                                          | 9080              | Sert le contenu du dossier <i>public/</i> et exécute les scripts PHP. C'est le cœur applicatif.              |
| <b>mysql</b>      | Base de données MySQL 8.0                                         | 3306 (non exposé) | Conteneur de persistance pour les données métiers. En production, ce port n'est jamais exposé à l'extérieur. |
| <b>phpmyadmin</b> | Interface d'administration MySQL (usage développement uniquement) | 8082              | Permet d'interagir facilement avec la base pendant les phases de test. Doit être désactivé en production.    |

Chaque conteneur est défini dans le fichier *docker-compose.yml*, qui décrit le réseau interne *mediastock-network* utilisé pour relier les services entre eux.

Ainsi, le conteneur web communique avec la base de données via le *hostname* *mysql* sans qu'aucun port ne soit ouvert vers l'extérieur.

### Justification

L'utilisation de Docker Compose permet de standardiser la configuration du projet sur toutes les machines de développement, et d'obtenir :

- une isolation complète des dépendances (Apache, PHP, MySQL, phpMyAdmin);
- une reproductibilité totale de l'environnement;
- un démarrage immédiat sans installation manuelle de serveurs locaux;
- la possibilité d'utiliser des versions exactes des images officielles (pas d'incohérence entre les postes).

## 3.2 Volumes et persistance

- **Bind mount :**

Le montage `./var/www/html` lie le répertoire courant du projet au répertoire du serveur Apache dans le conteneur.

Cela permet le *hot-reload* du code sans avoir à reconstruire l'image à chaque modification (gain de temps en développement).

- **Volume nommé :**

Le volume *mysql-data* est utilisé pour stocker les données MySQL de manière persistante, même après un arrêt ou une suppression des conteneurs.

Cela évite de perdre la base de données entre deux sessions de développement.

### Avantages de cette approche :

- **Séparation claire** entre code applicatif et données persistées.
- **Reproductibilité garantie** : le code est versionné, la donnée est conservée localement dans un volume dédié.
- **Maintenance facilitée** : les volumes peuvent être sauvegardés, purgés ou réinitialisés indépendamment du code.

## 3.3 Justification de l'utilisation de Docker

- **Cohérence entre environnements** : le même fichier *docker-compose.yml* est utilisé en développement, en test et en production, garantissant un comportement identique sur toutes les machines.
- **Isolation des services** : chaque composant (PHP/Apache, MySQL, phpMyAdmin) fonctionne dans son propre conteneur, ce qui évite les conflits de dépendances et renforce la sécurité.
- **Portabilité** : le projet peut être lancé sur n'importe quel poste disposant de Docker, sans prérequis supplémentaires (ni PHP ni MySQL installés localement).
- **Maintenance et scalabilité** : en production, il devient possible de faire évoluer chaque service indépendamment (mise à jour PHP, migration de base).

### 3.4 Environnement de production

Le projet a été déployé en production sur la plateforme d'hébergement *Alwaysdata*, qui permet d'exécuter une application PHP avec base de données MySQL dans un environnement mutualisé.

Dans ce contexte, l'environnement de production diffère de l'environnement de développement local :

- en développement, l'application est exécutée dans des conteneurs Docker,
- en production, l'application est hébergée directement sur l'infrastructure Alwaysdata, sans utilisation de Docker Compose.

*Le déploiement consiste à :*

1. exporter ou adapter les fichiers nécessaires depuis le dépôt GitHub,
2. téléverser le code source sur l'espace d'hébergement Alwaysdata,
3. configurer la base de données MySQL fournie par l'hébergeur,
4. adapter les paramètres de connexion dans le fichier de configuration prévu à cet effet,
5. vérifier le bon fonctionnement de l'application en ligne.

L'environnement Alwaysdata fournit directement :

- l'hébergement web Apache/PHP,
- la base de données MySQL,
- l'accès sécurisé à l'application via HTTPS,
- les outils d'administration nécessaires au déploiement.

#### Justification

Le choix d'Alwaysdata permet de simplifier la mise en production, sans avoir à administrer un serveur complet. Cette solution est adaptée à un projet pédagogique, car elle offre un environnement stable, compatible avec PHP/MySQL, et facilement accessible pour le déploiement d'une application web dynamique.

## 4. Backend (PHP + PDO)

Le backend de *MédiaStock* repose sur un socle PHP 8.2 utilisant PDO (PHP Data Objects) pour interagir avec la base de données MySQL 8.0.

Cette couche constitue le cœur de l'application : elle gère les traitements métier, la communication avec la base, et expose des *endpoints* simples consommés par le frontend via des requêtes AJAX (fetch).

### 4.1 Connexion et configuration de la base de données

#### Classe Database.php

Le projet *MédiaStock* utilise une classe dédiée à la gestion de la connexion à la base de données MySQL, nommée Database, située dans le namespace Config.

Cette classe centralise l'accès à la base de données et repose sur le pattern Singleton, garantissant qu'une seule connexion PDO est instanciée et partagée dans toute l'application.

***La connexion est établie via l'extension PDO, ce qui permet :***

- une abstraction du moteur de base de données,
- l'utilisation de requêtes préparées,
- une meilleure sécurité face aux injections SQL.

Les paramètres de connexion (hôte, nom de la base, utilisateur, mot de passe) ne sont pas écrits en dur dans le code. Ils sont chargés depuis un fichier de configuration externe *env.php*, ce qui facilite la maintenance et le travail en équipe.

Dans le contexte de développement local, l'hôte de la base de données est défini sur *mysql*, correspondant au nom du service MySQL déclaré dans le fichier *docker-compose.yml*.

Cette approche est indispensable dans un environnement conteneurisé, où *localhost* ne permet pas de joindre un autre conteneur.

Lors de l'instanciation, la classe :

- initialise la connexion PDO avec l'encodage *utf8mb4*,
- active le mode d'erreur par exceptions,
- configure le retour des résultats sous forme de tableaux associatifs,
- désactive l'émulation des requêtes préparées pour un comportement plus fiable côté serveur.

En cas d'échec de connexion, une exception est interceptée afin d'éviter l'exposition d'informations sensibles, conformément aux bonnes pratiques de sécurité. La gestion détaillée des erreurs est réservée à un contexte de développement.

La méthode statique *getInstance()* permet d'accéder à l'instance unique de la classe, tandis que la méthode *getConnection()* retourne l'objet PDO prêt à être utilisé par les différentes couches de l'application (modèles, services, etc.).

### **Fichier de configuration env.php**

Le fichier *env.php* contient les paramètres de connexion à la base de données pour l'environnement local Docker.

Il retourne un tableau associatif listant explicitement les clés de configuration attendues par l'application (*DB\_HOST*, *DB\_NAME*, *DB\_USER*, *DB\_PASSWORD*).

Ce fichier sert de référence commune pour les développeurs, garantissant une configuration homogène du projet en environnement de développement.

Il permet également d'isoler les informations sensibles du code applicatif et de préparer une future adaptation vers un environnement de production, sans modification de la logique métier.

### **Justification**

- PDO a été choisi plutôt que *mysqli* pour sa flexibilité, son support multi-SGBD et ses fonctionnalités avancées (requêtes préparées, transactions, gestion d'erreurs unifiée).
- La centralisation de la configuration dans *config/database.php* garantit une maintenance simple : en cas de changement d'hôte ou de mot de passe, une seule modification suffit.



- L'utilisation de variables de configuration pour la base de données renforce la sécurité, en évitant la présence d'informations sensibles directement dans le code source.

## 4.2 Organisation du code et accès aux données

### Structure des fichiers

Le code backend est organisé selon une logique inspirée du modèle MVC simplifié :

```
src/
└── models/      # Classes métier (Item, Emprunteur, Pret, etc.)
```

Les modèles représentent les entités du domaine (item, prêt, emprunteur...).

### Endpoints PHP (API légère)

Les endpoints sont regroupés dans **public/api/**, et chacun a une responsabilité unique, par exemple :

| Endpoint                  | Rôle                                                          |
|---------------------------|---------------------------------------------------------------|
| <i>additem.php</i>        | Crée un nouvel item dans la base et renvoie son ID.           |
| <i>updateitem.php</i>     | Met à jour les informations d'un item (p. ex : lien QR code). |
| <i>addemprunteur.php</i>  | Enregistre un nouvel emprunteur si nécessaire.                |
| <i>addpret.php</i>        | Crée un prêt actif (item + emprunteur + dates).               |
| <i>cloturepret.php</i>    | Termine un prêt actif et met à jour l'état du matériel.       |
| <i>getactiveloans.php</i> | Récupère la liste des prêts en cours.                         |

Chaque endpoint :

- Reçoit des données via `$_POST` ou `$_GET`.
- Valide les entrées (présence, type, valeurs autorisées).
- Appelle la couche repository correspondante.
- Retourne une réponse JSON structurée (succès / échec + données associées).

### Flux typiques de l'application

#### 1. Ajout d'un matériel

- *additem.php* crée un nouvel enregistrement dans la table item et renvoie l'ID.
- *updateitem.php* associe ensuite un QR code à cet ID pour l'impression.

#### 2. Création d'un prêt

- *addemprunteur.php* vérifie si la personne existe déjà ; sinon, elle est ajoutée.
- *addpret.php* enregistre le prêt et change l'état de l'item à "indisponible".

#### 3. Restitution d'un prêt

- Vérification du prêt actif correspondant.
- *cloturepret.php* clôture le prêt et repasse l'item à l'état "disponible".

#### 4. Modification d'un matériel

→ *updateitem.php* permet de mettre à jour dynamiquement un ou plusieurs champs d'un matériel existant dans la base (p.ex. nom, modèle, image\_url, état, catégorie).

### 4.3 Justification et bonnes pratiques backend

- **Endpoints indépendants et lisibles** : chaque fichier PHP a une responsabilité claire et peut être testé séparément.
- **Sécurité renforcée** : toutes les requêtes passent par des statements préparés, empêchant les injections SQL.
- **Schéma relationnel bien défini** : cohérence des relations (clés étrangères, contraintes d'intégrité), facilitant la maintenance.
- **Réponses JSON standardisées** : simplifient l'intégration avec le frontend via *fetch()*.
- **Découplage logique / interface** : la logique métier reste côté serveur, ce qui facilite les évolutions futures (ex. migration vers une API REST complète ou un framework MVC comme Laravel).

## 5. Base de données

### 5.1 Schéma relationnel général

La base de données *MySQL 8.0* constitue le cœur du système de persistance de **MédiaStock**.

Elle a été conçue selon un modèle relationnel normalisé (3NF) pour assurer la cohérence, la performance et la maintenabilité des données.

Le modèle repose sur six entités principales :

| Table             | Rôle                                                                                                                                     |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ITEM</b>       | Contient les informations relatives à chaque matériel (nom, modèle, QR code unique, état, disponibilité, catégorie).                     |
| <b>PRET</b>       | Historise ensemble des prêts et restitutions de matériel (dates, emprunteur, administrateur prêteur, état du matériel au départ/retour). |
| <b>EMPRUNTEUR</b> | Gère les utilisateurs qui empruntent le matériel (nom, prénom, formation, statut actif/archivé).                                         |
| <b>ADMIN</b>      | Contient les comptes d'administration avec identifiant et mot de passe haché (authentification et sécurité).                             |
| <b>FORMATION</b>  | Représente les différentes formations de l'établissement, associées aux emprunteurs étudiants.                                           |
| <b>CATEGORIE</b>  | Classifient les matériels selon leur type.                                                                                               |

Une indexation ciblée (sur *id\_item*, *id\_emprunteur*, *id\_pret*, *id\_cat*) a été mise en place pour accélérer les requêtes les plus fréquentes (recherche par catégorie, par disponibilité et par état).

Le MCD et MLD sont disponibles en annexe du cahier des charges, présentant graphiquement les relations entre ces tables.

## 5.2 Principales relations et contraintes

Les relations entre les entités de la base de données MediaStock sont directement issues du Modèle Conceptuel de Données (MCD) et sont implémentées à l'aide de clés étrangères afin de garantir l'intégrité référentielle.

### ***ITEM ↔ CATEGORIE***

- Un ITEM appartient à une et une seule CATEGORIE (1,1).
- Une CATEGORIE peut regrouper zéro à plusieurs ITEM (0,n).

Cette relation permet de classer chaque matériel dans une catégorie unique, facilitant l'organisation de l'inventaire et le filtrage des matériels.

### ***ITEM ↔ PRÊT***

- Un PRÊT concerne un seul ITEM (1,1).
- Un ITEM peut être associé à plusieurs PRÊTS successifs dans le temps (0,n).

La clé étrangère *item\_id* dans la table *pret* permet de conserver l'historique complet des prêts pour chaque matériel, tout en garantissant qu'un prêt ne concerne qu'un seul item à la fois.

### ***EMPRUNTEUR ↔ PRÊT***

- Un PRÊT est effectué par un seul EMPRUNTEUR (1,1).
- Un EMPRUNTEUR peut réaliser plusieurs PRÊTS (0,n).

Cette relation assure la traçabilité des prêts par emprunteur.

La suppression physique d'un emprunteur est évitée afin de préserver l'historique des prêts ; un archivage logique est privilégié.

### ***ADMINISTRATEUR ↔ PRÊT***

- Un PRÊT est enregistré par un seul ADMINISTRATEUR (1,1).
- Un ADMINISTRATEUR peut gérer plusieurs PRÊTS (0,n).

Cette relation permet d'identifier l'administrateur responsable de chaque opération de prêt ou de restitution, renforçant la traçabilité des actions.

### ***EMPRUNTEUR ↔ FORMATION***

- Un EMPRUNTEUR peut être rattaché à zéro ou une FORMATION (0,1).
- Une FORMATION peut regrouper plusieurs EMPRUNTEURS (0,n).

Cette association permet de distinguer les étudiants, rattachés à une formation, des intervenants externes, qui ne sont associés à aucune formation.

### 5.3 Décisions de conception notables

- **formation\_id nullable** pour permettre d’ajouter des intervenants externes non liés à une formation interne.
- **Archivage logique** avec un champ booléen *archived* dans les tables item et emprunteur. Cela permet de masquer un enregistrement sans le supprimer, préservant l’historique des prêts.
- **Utilisation d’un champ statut** (ex. “disponible”, “indisponible”, “en retard”) pour le suivi du matériel en temps réel.
- **Données de référence initiales** (catégories, formations, compte admin par défaut) insérées via le script *sql/init.sql* exécuté automatiquement lors du premier build Docker.
- **Encodage UTF-8 MB4** pour supporter tous les caractères spéciaux et émojis, garantissant la compatibilité multilingue.

### 5.4 Choix techniques et justification

| Aspect            | Choix                                           | Justification                                                                                                        |
|-------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| SGBD              | MySQL 8.0                                       | Système mature, léger, parfaitement intégré à PHP/PDO, adapté au volume estimé (≈100 matériels, 10–20 prêts/semaine) |
| SQL direct        | SQL via PDO                                     | Plus simple pour un projet éducatif et garantit un contrôle total sur les requêtes et transactions.                  |
| Encodage          | utf8mb4                                         | Assure la compatibilité universelle des données (accents, symboles, émojis).                                         |
| Transactions      | BEGIN/COMMIT/ROLLBACK                           | Empêche les incohérences entre <i>pret</i> et <i>item</i> lors des prêts simultanés.                                 |
| Archivage logique | Champ booléen archived                          | Préserve la traçabilité, conforme aux exigences de suivi et aux règles RGPD.                                         |
| Indexation        | Sur les clés étrangères et colonnes de filtrage | Accélère les recherches par catégorie, disponibilité, état sans ralentir les insertions.                             |

### 5.5 Sécurité et intégrité des données

- **Requêtes préparées PDO** systématiques (protection contre les injections SQL).
- **Contraintes de clés étrangères** (FOREIGN KEY) actives pour éviter les enregistrements orphelins.
- **Mot de passe admin stocké haché** (bcrypt) dans la table ADMIN.
- **Suppression logique** au lieu de physique pour assurer la traçabilité complète de l’activité.
- **Politique RGPD respectée**: collecte minimale des données (nom, prénom, formation uniquement) et possibilité de suppression sur demande.

## 5.6 Justification d'ensemble

La conception relationnelle de la base de données de *MédiaStock* a été pensée pour :

1. **Garantir la cohérence et la traçabilité** : chaque prêt conserve le lien avec l'emprunteur, l'administrateur et l'item prêté.
2. **Préserver l'historique des opérations** sans altérer les données sources (archivage logique).
3. **Faciliter la maintenance** grâce à un schéma clair, bien indexé et cohérent avec les flux applicatifs.
4. **Assurer la performance et la sécurité**, notamment via PDO, l'encodage « UTF-8 MB4 » et les contraintes relationnelles.
5. **Préparer les évolutions futures**, comme les statistiques de prêt ou les rapports d'utilisation par catégorie.

## 6. Frontend

### 6.1 Technologies utilisées

Le frontend de *MédiaStock* repose sur une architecture légère, 100 % web, pensée pour être mobile-first, simple à maintenir et performante sur les appareils administratifs (smartphones, tablettes, PC portables).

*Technologies principales :*

- **HTML5** : structure sémantique des pages, conforme aux standards modernes.
- **CSS3 + Bootstrap 5.3** : design responsive, système de grilles, composants pré-stylés (modales, boutons, formulaires).
- **JavaScript ES6+** : logique côté client (gestion d'événements, validation, appels API via fetch).
- **Flatpickr** : plugin de sélection de dates ergonomique et compatible mobile.
- **FontAwesome** : icônes vectorielles utilisées pour renforcer la lisibilité et l'ergonomie.

Ces technologies permettent une interface claire, fluide et universellement compatible, sans dépendance à un framework lourd (comme React ou Vue), ce qui simplifie la maintenance et le déploiement.

### 6.2 Structure et principales pages

Le dossier *public/* contient l'ensemble des ressources accessibles par le navigateur.

Les fichiers HTML ou PHP sont accompagnés de leurs scripts et feuilles de style correspondants :

| Page                      | Fonction principale                                                                                                                                                         |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>index.php</b>          | Tableau de bord principal : affiche la liste des matériels, filtrables par catégorie, disponibilité et par état. Accès direct aux détails via un panneau <i>offcanvas</i> . |
| <b>creation-pretp.php</b> | Interface de création d'un prêt : sélection d'un matériel via QR code disponible, choix de l'emprunteur et des dates de sortie/retour (Flatpickr).                          |

| Page                         | Fonction principale                                                                                                  |
|------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>restitution.php</b>       | Restitution d'un prêt actif par scan du QR code. Mise à jour de l'état du matériel et clôture du prêt en temps réel. |
| <b>matériel.php</b>          | Ajout d'un nouvel item dans l'inventaire avec génération et téléchargement du QR code correspondant                  |
| <b>modification-item.php</b> | Formulaire complet permettant la mise à jour d'un item existant (nom, modèle, image, état, catégorie)                |
| <b>accueil.html</b>          | Authentification administrateur avec gestion de session PHP.                                                         |

Chaque page charge dynamiquement les données depuis les *endpoints* PHP situés dans *public/api/*, grâce à des appels *fetch()* asynchrones.

Les réponses JSON sont ensuite injectées dans le DOM via JavaScript, permettant une interface fluide sans rechargement complet de page.

### 6.3 Comportements interactifs et ergonomie

- **Filtrage dynamique** : les listes de matériels se mettent à jour instantanément selon la catégorie, la disponibilité et l'état.
- **Modales de confirmation Bootstrap** : utilisées pour valider les actions sensibles (suppression, prêt, restitution).
- **Calendriers intuitifs** : *Flatpickr* permet une sélection fluide et fiable des dates de prêt et de retour (format ISO standard pour compatibilité serveur).
- **Lecture et génération de QR Codes** : réalisée côté client, via des bibliothèques JavaScript (*html5-qrcode*), ce qui évite toute dépendance externe et accélère les opérations de terrain.

*Une attention particulière a été portée à l'ergonomie mobile :*

- utilisation de grands boutons tactiles,
- affichage simplifié sur petits écrans,
- navigation linéaire (haut → bas),
- couleurs contrastées et cohérentes (palette Bootstrap + accent couleur rouge *MédiaStock*).

### 6.4 Justification des choix

| Composant              | Raison du choix                                                                             |
|------------------------|---------------------------------------------------------------------------------------------|
| <b>Bootstrap 5.3</b>   | Gain de temps de développement, responsive design natif, cohérence visuelle multi-écrans.   |
| <b>Flatpickr</b>       | Saisie fiable et rapide des dates, meilleure UX sur mobile qu'un champ texte HTML standard. |
| <b>FontAwesome</b>     | Renforce la compréhension visuelle (ex. icône de prêt, retour, QR code).                    |
| <b>JavaScript ES6+</b> | Code clair, moderne et compatible sur la majorité des navigateurs actuels.                  |

| Composant      | Raison du choix                                                                        |
|----------------|----------------------------------------------------------------------------------------|
| QR côté client | Rapidité et autonomie en mobilité (aucune requête serveur nécessaire pour la lecture). |

En résumé, le frontend a été pensé pour offrir une expérience fluide, intuitive et réactive, adaptée à un usage régulier sur le terrain (emprunts et restitutions rapides).

## 7. QR Codes

### 7.1 Rôle et fonctionnement

Les QR codes sont au cœur du fonctionnement de *MédiaStock*.

Chaque matériel est identifié par un QR unique, servant de lien direct entre l'objet physique et sa fiche numérique dans la base de données.

Ce système permet :

- de créer et imprimer les étiquettes lors de l'ajout d'un nouveau matériel,
- de scanner rapidement un QR code pour lancer une opération (prêt, restitution, consultation),

### 7.2 Génération des QR Codes

Une approche de côté client en JavaScript a été prévue pour la génération des QR :

- Génération immédiate à l'ajout d'un nouvel item via une librairie telle que `QRCode.js`.
- L'utilisateur peut afficher, télécharger ou imprimer le QR directement depuis le navigateur.
- Ce QR code encode un identifiant unique, souvent sous la forme :

*`http://localhost:9080/materiel.php?code=42`*

ou simplement l'ID numérique de l'item (42), selon les besoins du mode hors-ligne.

Les QR codes sont ensuite intégrés aux fiches PDF d'inventaire ou imprimés sous forme d'étiquettes autocollantes.

### 7.3 Lecture et utilisation

- Le scan s'effectue directement depuis le navigateur mobile, grâce à *html5-qrcode*. Ces bibliothèques exploitent l'API `getUserMedia()` pour accéder à la caméra du smartphone.
- Lors du scan :
  - le QR est décodé,
  - l'ID de l'item est extrait,
  - une requête `fetch()` est envoyée vers l'API (`findByQrCode.php?code=...`) pour charger la fiche correspondante.

Ce mode de fonctionnement évite toute application tierce et permet une utilisation immédiate sur le terrain (ex. scan d'un ordinateur prêté pour enregistrer sa restitution).

## 7.4 Convention d'identification

Chaque QR code correspond à l'identifiant unique de l'item dans la base (id\_item).

Ainsi :

- pas de doublon,
- lien direct entre QR et matériel physique,
- opération unifiée pour prêt/restitution.

Cette convention garantit la traçabilité totale et la rapidité des opérations, tout en simplifiant le code côté backend (pas de recherche par nom ou par code alphanumérique).

## 7.5 Justification et bénéfices

- **Rapidité** : un scan suffit pour accéder ou modifier un enregistrement (aucune saisie manuelle).
- **Simplicité technique** : pas d'application mobile dédiée — tout fonctionne via le navigateur.
- **Fiabilité** : le QR code servant de clé primaire physique, les erreurs humaines sont quasi nulles.
- **Évolutivité** : la génération peut être étendue à d'autres supports (PDF d'inventaire, affichage dynamique, statistiques).

En définitive, le QR code est l'élément pivot du modèle *MédiaStock*, garantissant à la fois ergonomie, performance et traçabilité.

# 8. Sécurité, conformité et sessions

## 8.1 Sécurisation des communications

En production, l'application est hébergée sur *Alwaysdata* et accessible en HTTPS, ce qui permet de chiffrer les échanges entre le navigateur client et le serveur.

Cela garantit la confidentialité des données transmises, notamment :

- les identifiants de connexion,
- les informations liées aux emprunteurs,
- les actions effectuées sur les prêts et restitutions.

L'utilisation du protocole HTTPS permet également de limiter les risques d'interception des données lors des communications réseau.

### Justification

Le chiffrement des échanges est indispensable pour sécuriser l'accès à une application de gestion manipulant des données nominatives. Dans le cadre de l'hébergement sur *Alwaysdata*, cette sécurisation est prise en charge par la plateforme, ce qui simplifie le déploiement tout en assurant un niveau de protection adapté.



## 8.2 Authentification et gestion de sessions

- **Authentification administrateur :**

L'authentification repose sur un mécanisme PHP natif, sans framework externe, afin de garantir la simplicité, la portabilité et la compatibilité avec l'environnement Dockerisé.

Les mots de passe sont hachés avec *bcrypt* à l'aide des fonctions *password\_hash()* et *password\_verify()*, empêchant toute compromission de mot de passe en clair dans la base de données.

- **Création de session utilisateur :**

Lors d'une connexion réussie, une session PHP est initialisée via *session\_start()*.

La variable *\$\_SESSION['username']* conserve l'identifiant de l'administrateur connecté.

Ce mécanisme permet de maintenir l'état de connexion tout au long de la navigation.

- **Vérification de session à chaque accès :**

Chaque page sécurisée (tableau de bord, gestion de prêts, inventaire) intègre un contrôle de session via un script dédié (*login\_verify.php*).

Ce script s'exécute à chaque chargement et vérifie :

- **Si une session est active**, sinon redirection vers la page publique *accueil.html*.
- **Si la session est expirée**, selon une durée d'inactivité maximale fixée à 5 minutes (300 secondes).  
Au-delà, la session est détruite et l'utilisateur redirigé vers la page de connexion (*accueil.html*).

Exemple du code de vérification :

```
<?php
session_start();

// durée maximale d'inactivité en secondes (5 min = 300s)
$session_timeout = 300;

// Vérifie si l'utilisateur est connecté
if (!isset($_SESSION['username']) || empty($_SESSION['username'])) {
    header('Location: accueil.html');
    exit();
}

// Vérifie si la session a expiré
if (isset($_SESSION['last_activity']) && (time() - $_SESSION['last_activity']) >
$session_timeout) {
    // Destruction de la session et redirection
    session_unset();
    session_destroy();
    header("Location: index.php");
    exit;
```

```

}

// Mise à jour de l'heure de dernière activité
$_SESSION['last_activity'] = time();
?>

```

Ce système permet :

- d’empêcher tout accès non autorisé aux pages internes ;
- d’assurer la déconnexion automatique après 5 minutes d’inactivité ;
- de mettre à jour la dernière activité à chaque action utilisateur.

- **Protection des cookies de session :**

- *HttpOnly activé* : empêche tout accès JavaScript au cookie (protection XSS).
- *Secure activé* : le cookie n’est transmis qu’en HTTPS.

- **Déconnexion explicite :**

L’utilisateur peut se déconnecter via un bouton “*Déconnexion*”, qui :

- détruit la session serveur (*session\_destroy()*),
- invalide les cookies de session côté client,
- et redirige vers la page publique d’accueil (*accueil.html*).

*Cette fonctionnalité pourra être étendue ultérieurement à une gestion multi-comptes ou à des profils différenciés (ex. administrateur, technicien, intervenant).*

### 8.3 Protection contre les attaques web

| Type d’attaque                           | Mesure préventive                                          | Description / justification                                                                                   |
|------------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Injection SQL</b>                     | Requêtes préparées PDO                                     | Empêche l’injection de commandes malveillantes dans les champs utilisateur.                                   |
| <b>XSS (Cross-Site Scripting)</b>        | Échappement des sorties HTML ( <i>htmlspecialchars()</i> ) | Aucune donnée utilisateur n’est réinjectée brute dans le DOM.                                                 |
| <b>CSRF (Cross-Site Request Forgery)</b> | Jetons CSRF uniques dans les formulaires sensibles         | Un token aléatoire est stocké en session et validé à chaque soumission.                                       |
| <b>Brute-force login</b>                 | Temporisation / verrouillage progressif                    | Après plusieurs échecs, délai imposé avant nouvelle tentative. ( <i>prévu pour une évolution ultérieure</i> ) |

Ces mesures combinées réduisent significativement la surface d’attaque et garantissent un niveau de sécurité adapté à un environnement de production exposé au web.

### 8.4 Sécurité des environnements Docker

En environnement de développement, Docker permet d’isoler les différents services utilisés par l’application, notamment le serveur web, la base de données MySQL et phpMyAdmin.

***Les précautions suivantes ont été mises en place :***

- le conteneur MySQL communique uniquement avec le conteneur web via le réseau Docker interne,
- phpMyAdmin est réservé à l'environnement de développement,
- les fichiers de configuration sensibles sont exclus du dépôt Git,
- les variables de connexion sont séparées du code applicatif.

En production, ces mécanismes ne sont plus gérés via Docker, car l'application est hébergée sur *Alwaysdata* dans un environnement mutualisé. La sécurité de l'infrastructure de production repose donc sur les mécanismes fournis par l'hébergeur.

**Justification**

Docker a été utilisé pour fiabiliser et standardiser l'environnement local de développement. En revanche, l'hébergement en production sur Alwaysdata repose sur une logique différente, plus simple à administrer dans le cadre d'un projet web PHP/MySQL.

## **8.5 Conformité RGPD**

- **Collecte minimale** : uniquement les informations strictement nécessaires (nom, prénom, formation, identifiant interne).
- **Rétention limitée** : les données sont conservées uniquement le temps nécessaire à la gestion des prêts, puis archivées pour assurer la traçabilité.
- **Droit d'accès / suppression** : un emprunteur peut demander la consultation ou la suppression de son compte, conformément au droit à l'effacement. *Cette fonctionnalité pourra être étendue ultérieurement.*
- **Archivage logique** : les emprunteurs ou items supprimés sont simplement marqués comme archived=1, assurant la traçabilité tout en permettant leur anonymisation ultérieure.

Ces mesures respectent le principe de minimisation et les obligations de traçabilité imposées par la réglementation RGPD.

## **8.6 Justification globale**

La politique de sécurité de *MédiaStock* vise à réduire la surface d'attaque au strict minimum tout en respectant les normes de confidentialité et de conformité.

Les choix techniques — pas de phpMyAdmin en production, non-exposition des ports de base de données, et gestion robuste des sessions — garantissent un niveau de protection élevé pour une application de gestion de matériel utilisée en environnement scolaire et administratif.

## 9. Déploiement et exploitation

### 9.1 Procédure d'installation (environnement de développement)

#### 1. Configuration initiale

- Copier le fichier modèle : `cp .env.example .env` et renseigner les variables d'environnement (*base MySQL, identifiants, ports, chemins*).
- Les variables par défaut permettent une installation locale immédiate.

#### 2. Démarrage de la stack Docker

`docker-compose up -d --build`

Cette commande construit les images et démarre les services :

- web → Apache/PHP sur port 9080
- mysql → base de données MySQL
- phpmyadmin → accessible sur 8082 (uniquement en dev)

#### 3. Vérification

- `docker compose ps` → liste les conteneurs actifs
- `docker compose logs -f web/mysql` → consulte les logs en direct
- Vérifier :
  - `http://localhost:9080` → application web
  - `http://localhost:8082` → phpMyAdmin

#### 4. Initialisation de la base

- Si besoin, exécuter importer manuellement le script `sql/init.sql` via phpMyAdmin.

#### 5. Hot reload

- Les modifications du code sont automatiquement visibles (*bind mount actif*), sans reconstruire les images.

### 9.2 Déploiement en production

#### 1. Préparation

- Se connecter à l'interface d'administration Alwaysdata.
- Créer une base de données MySQL via le panneau de gestion.
- Adapter le fichier `config/env.php` avec les paramètres de connexion fournis par Alwaysdata (hôte, nom de la base, utilisateur, mot de passe). Ce fichier ne doit pas être versionné.

#### 2. Déploiement du code

- Téléverser le code source sur l'espace d'hébergement via FTP/SFTP ou Git.
- Vérifier que le DocumentRoot Apache pointe bien vers le dossier `public/`.
- S'assurer que le fichier `.htaccess` est bien présent et actif.

#### 3. Initialisation de la base de données

- Importer le script `sql/init.sql` via l'outil d'administration MySQL fourni par Alwaysdata (phpMyAdmin ou équivalent).
- Vérifier la présence des données initiales (catégories, formations, compte admin par défaut).

#### 4. *Vérification*

- Tester l'accès à l'application via l'URL publique fournie par Alwaysdata.
- Contrôler le bon fonctionnement du HTTPS (certificat géré automatiquement par la plateforme).
- Tester les fonctionnalités principales : connexion, ajout d'item, prêt, restitution.

#### 5. *Sauvegardes*

- Sauvegarde de la base MySQL via l'export phpMyAdmin ou l'outil de sauvegarde Alwaysdata.
- Stockage des sauvegardes sur un disque externe et/ou cloud sécurisé.

### 9.3 Justification des choix d'exploitation

| Principe                               | Objectif                                                   | Résultat attendu                                                                                                                                              |
|----------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Docker (environnement local)</i>    | Déploiement reproductibleReproductibilité du développement | Le projet peut être reconstruit à l'identique sur toute machine.L'environnement local peut être reconstruit à l'identique sur toute machine de développement. |
| <i>Hébergement Alwaysdata</i>          | Déploiement simplifié en production                        | Plateforme PHP/MySQL clé en main, sans administration serveur. Compatible avec l'architecture du projet.                                                      |
| <i>Fichier env.php (non versionné)</i> | Séparation code / paramètres sensibles                     | Les identifiants de connexion à la base de données ne sont jamais exposés dans le dépôt Git                                                                   |
| <i>HTTPS (géré par Alwaysdata)</i>     | Sécurité et conformité RGPD                                | Chiffrement des échanges assuré automatiquement par la plateforme, sans configuration manuelle.                                                               |
| <i>Sauvegarde</i>                      | Résilience                                                 | Restauration rapide après panne ou perte de données, via export phpMyAdmin ou outils Alwaysdata.                                                              |

### 9.4 Résumé

Grâce à sa structure conteneurisée et à sa documentation claire, le projet *MédiaStock* peut être installé, testé et déployé en production de manière rapide, fiable et sécurisée.

Le couple Docker + fichiers d'environnement garantit la reproductibilité complète, tandis que les procédures de sauvegarde assure la pérennité du système.

## 10. Bonnes pratiques de développement

### 10.1 Gestion du versionnement (Git)

- **Branches :**
  - *main* → version stable et déployable en production.
  - *Une branche par développeur* → permet d'éviter la "casse" de la branche principale et de travailler en parallèle sans conflits.
- **Workflow recommandé :**
  - Créer une branche à partir de *main*  
*git checkout -b nom\_de\_la\_personne*
  - Développer, tester et valider localement.
  - Créer une *merge request* vers *main*.
  - Après validation, fusionner dans *main* uniquement les versions testées et stables.
- **Conventional Commits :**

Chaque commit doit suivre une structure claire, par exemple :

  - *feat*: → ajout d'une nouvelle fonctionnalité.
  - *fix*: → correction de bug.
  - *docs*: → mise à jour de documentation.
  - *refactor*: → amélioration du code sans changement fonctionnel.
  - *chore*: → tâches techniques, dépendances, nettoyage.

#### Exemple :

*feat(api): ajout endpoint addpret.php avec contrôle de disponibilité*

*fix(frontend): correction de la sélection de date Flatpickr*

#### **Justification :**

Cette convention garantit un historique Git clair et homogène, facilite la traçabilité des évolutions, et permet une intégration continue plus efficace (*CI : Continuous Integration/CD: Continuous Deployment*).

Elle favorise également la collaboration entre développeurs et la compréhension rapide des changements apportés au code .

### 10.2 Gestion de l'environnement Docker

- **Rebuild sélectif :**

Ne reconstruire les images (docker compose build) que lorsque :

  - le Dockerfile ou les dépendances changent ou
  - de nouvelles librairies sont ajoutées.

Les modifications de code sont visibles instantanément via le *bind mount* (./var/www/html), sans rebuild.
- **Utilisation de .env :**
  - Le fichier *.env* contient les variables sensibles (mots de passe, ports, noms de DB).
  - Ne jamais le versionner → seul *.env.example* est présent pour référence.
  - En production sur Alwaysdata, ce fichier est remplacé par une configuration adaptée aux paramètres fournis par l'hébergeur, sans passer par Docker.

- **Commandes utiles (cheat-sheet) :**

|                                           |                                         |
|-------------------------------------------|-----------------------------------------|
| <code>docker-compose restart web</code>   | # Redémarrer le conteneur web           |
| <code>docker-compose down</code>          | # Stopper la stack (garde les volumes)  |
| <code>docker-compose down -v</code>       | # Stopper et supprimer les données      |
| <code>docker-compose exec web bash</code> | # Ouvrir un shell dans le conteneur web |
| <code>docker-compose logs -f web</code>   | # Suivre les logs du serveur Apache/PHP |

**Justification :**

Ces pratiques assurent un développement fluide, reproductible et conforme entre toutes les machines.

### 10.3 Tests et validation

- **Structure pyramidale :**

1. **Tests unitaires** → fonctions PHP critiques (connexion PDO, validation de champs, génération QR).
2. **Tests d'intégration** → enchaînement logique entre endpoints (*addpret*, *cloturepret*).
3. **Tests fonctionnels** → validation manuelle via interface web (cas de prêts, restitutions, erreurs).
4. **Tests de sécurité** → essais d'injection, XSS.

- **Cas d'usage simulés :**

1. Ajout d'un item dans la base de données
2. Prêt puis restitution d'un item existant.
3. Prêt simultané d'un même item (doit échouer).
4. Ajout d'un emprunteur.
5. Lecture de QR code sur mobile (test Chrome/Android et Safari/iOS).
6. Modification des caractéristiques d'un item

**Justification :**

La couverture de tests garantit la stabilité avant chaque fusion dans main.

### 10.4 Documentation et maintenance

- **Commentaires clairs** dans le code PHP et JS (en français ou anglais, mais cohérents).
- **README.md** à jour : procédures d'installation, schémas, ports et variables d'environnement.
- **Fiche technique** (ce document) jointe au dépôt pour faciliter la reprise par un nouveau développeur.

**Bon réflexe :**

- Avant chaque commit, vérifier les logs du serveur par « `docker-compose logs web` » pour s'assurer qu'aucune erreur PHP ou avertissement n'est présent.
- Après chaque pull (récupération du code d'un autre développeur), vider le cache du navigateur par `CTRL + F5` afin de forcer le rechargement des scripts et styles mis à jour.

## 11. Points d'attention et décisions techniques prises en cours de réalisation

### 11.1 Configuration d'Apache et sécurité du DocumentRoot

- **Docroot Apache** → **public/**

La configuration d'Apache a été modifiée directement dans l'image Docker (*sed* appliqué sur la configuration par défaut d'Apache) pour que le Document\_Root pointe vers le dossier *public/*.

- Ce choix permet :
  - de n'exposer que les fichiers front-end (HTML, CSS, JS, images) au navigateur,
  - de protéger le code applicatif (contenu dans *src/* et *config/*) contre tout accès direct,
  - d'adopter une structure standard et sécurisée des applications PHP modernes.

Cela améliore la sécurité tout en clarifiant la séparation entre les couches *frontend* et *backend*.

### 11.2 Organisation du backend : API centralisée

- Les endpoints PHP (scripts côté serveur répondant aux requêtes AJAX fetch) ont été regroupés dans le répertoire *public/api/*.

Apache ne sert donc que le dossier *public/*, qui contient :

- les fichiers d'interface (HTML, CSS, JS),
- le sous-dossier *api/* pour les requêtes asynchrones.

Cette organisation :

- améliore la lisibilité du projet et la modularité du code,
- simplifie les appels AJAX (structure d'URL claire et uniforme),
- garantit que seuls les endpoints nécessaires sont accessibles depuis le web.

Apache ne publie que le répertoire *public/*, protégeant le reste du code applicatif du projet.

```
public/
├── frontend/
└── api/
    ├── addpret.php
    ├── getactiveloads.php
    └── cloturepret.php
```

### 11.3 Base de données et intégrité applicative

- **Champ formation\_id nullable :**

Cette décision a été prise pour permettre la création d'intervenants externes ou de personnels sans rattachement direct à une formation.

Elle rend le modèle plus souple tout en respectant les règles d'intégrité référentielle.



- **Archivage logique (archived) :**

Les entités sensibles comme *item* et *emprunteur* disposent d'un *champ booléen archived*.

Lorsqu'un enregistrement est supprimé dans l'interface, il n'est pas effacé physiquement mais marqué comme inactif (*archived* = 1).

Ce mécanisme :

- préserve la traçabilité des prêts passés ;
- évite les ruptures de clés étrangères dans la table *pret* ;
- facilite la conformité RGPD (possibilité d'anonymiser ultérieurement les données).

Cette approche concilie sécurité, conformité et maintenance simplifiée.

## 11.4 Hébergement et contraintes techniques

- **GitHub Pages non adapté :** cette plateforme ne gère que les sites statiques (HTML/CSS/JS).

Comme *MédiaStock* repose sur un backend PHP/MySQL, un serveur d'exécution dynamique est indispensable.

- **Solutions recommandées pour la production :**

- **Alwaysdata :** plateforme d'hébergement mutualisé offrant PHP 8.x et MySQL 8.0, sans nécessiter l'administration d'un serveur complet. Elle fournit directement l'environnement Apache/PHP, la base de données MySQL, le HTTPS et les outils d'administration nécessaires au déploiement.

- **Autres options possibles (non retenues) :**

- VPS Dockerisé (ex. OVH, Hetzner, Scaleway) : déploiement complet et contrôle total sur la stack.
- Plateformes cloud (Render, Railway, Fly.io) : prennent en charge Docker Compose et les bases MySQL persistantes.

Alwaysdata a été privilégié pour sa simplicité de mise en œuvre dans un contexte pédagogique, tout en assurant un environnement stable, compatible et accessible.

## 11.5 Synchronisation et dépendances Docker

- **Problème constaté :** en développement, le conteneur phpMyAdmin démarrait parfois avant MySQL, causant une erreur de connexion.

- **Solutions envisagées :**

- ajout d'une directive *depends\_on* dans *docker-compose.yml* pour ordonner le démarrage ;
- simple rafraîchissement manuel de la page phpMyAdmin une fois MySQL prêt.

Ce problème est uniquement présent en environnement de développement local. En production sur Alwaysdata, il ne se pose pas : la plateforme gère directement la disponibilité de la base de données MySQL, sans Docker.

## 11.6 Synthèse des décisions techniques

| Décision                         | Objectif principal    | Impact positif                                                    |
|----------------------------------|-----------------------|-------------------------------------------------------------------|
| <i>Docroot limité à public/</i>  | Sécurité              | Aucune exposition du code backend au public                       |
| <i>API dans public/api/</i>      | Organisation claire   | Endpoints regroupés et facilement repérables                      |
| <i>formation_id nullable</i>     | Flexibilité           | Gestion d'utilisateurs non liés à une formation                   |
| <i>Archivage logique</i>         | Traçabilité           | Historique complet sans perte de données                          |
| <i>Hébergement Alwaysdata</i>    | Déploiement simplifié | PHP/MySQL clé en main, HTTPS intégré, sans administration serveur |
| <i>phpMyAdmin réservé au dev</i> | Sécurité              | Moins de surface d'attaque en production                          |
| <i>depends_on Docker</i>         | Stabilité dev         | Évite les erreurs d'initialisation MySQL                          |

## 11.7 Bilan global

Ces décisions, prises au fil du développement, ont permis de construire une application plus sécurisée, modulaire et facilement déployable.

L'architecture actuelle garantit la traçabilité complète des opérations, la portabilité Docker entre les environnements et la pérennité du code pour les futures équipes de maintenance.

# 12. Annexes utiles pour la reprise du projet

## 12.1 Cahier des charges détaillé

Contient :

- le contexte fonctionnel du projet ;
- les exigences techniques et sécuritaires (authentification, traçabilité, archivage, RGPD) ;
- le schéma de la base de données (MCD/MLD) ;
- les règles de déploiement et de sauvegarde ;
- le planning de développement (phases, livrables, jalons).

## 12.2 Fiche de réalisation

Document de synthèse destiné à la soutenance du projet, comprenant :

- les **ressources mobilisées** (Docker, PHP, MySQL, Bootstrap, JS, Git) ;
- les **accès et environnements** utilisés (localhost, phpMyAdmin, Docker Compose) ;
- les **livrables produits** :
  - code source GitHub,
  - base de données initialisée,

- conteneurs Docker prêts à l'emploi,
- documentation technique (présente fiche).

## 12.3 Utilité de ces annexes pour la reprise

Ces documents constituent le socle documentaire du projet.

Ils permettent à toute nouvelle équipe :

- de comprendre rapidement les choix techniques et fonctionnels ;
- de reconstruire l'environnement complet à partir de zéro ;
- de maintenir et faire évoluer l'application dans le respect des décisions d'origine ;
- de garantir la continuité pédagogique et technique du projet *MédiaStock*.

## 13. “Pourquoi ces choix ?” — Synthèse des décisions techniques

### 13.1 Docker : cohérence et portabilité

L'usage de Docker Compose a été retenu pour garantir une cohérence totale de l'environnement de développement entre tous les membres de l'équipe. Chaque développeur dispose ainsi d'une stack identique : Apache + PHP + MySQL + phpMyAdmin

Avantages principaux :

- **Installation rapide** : une seule commande *docker-compose up -d --build* permet de déployer l'environnement complet.
- **Isolation claire** : chaque service (web, base de données, outils) tourne dans un conteneur distinct, limitant les conflits de dépendances.
- **Portabilité** : la *stack* est reproductible sur tout système (Windows, macOS, Linux) garantissant un comportement identique entre les postes.
- **Maintenance facilitée** : mise à jour, suppression ou remplacement d'un service sans impacter les autres.

Docker est utilisé **uniquement en développement**. En production, l'application est hébergée sur **Alwaysdata**, qui fournit directement l'environnement PHP/MySQL sans nécessiter de conteneurisation.

Ce choix répond aux exigences du projet en matière de simplicité d'installation et d'homogénéité entre les postes de développement, tout en restant compatible avec un déploiement sur une plateforme d'hébergement mutualisé comme Alwaysdata.

### 13.2 PHP + MySQL : simplicité, fiabilité et compatibilité

Le couple **PHP/MySQL** a été choisi pour sa large documentation et son interopérabilité naturelle.

Il s'agit d'une solution éprouvée, parfaitement adaptée aux applications de gestion à charge modérée.

#### Raisons du choix :

- **Écosystème stable et maîtrisé** : technologie enseignée et largement utilisée.
- **Faible courbe d'apprentissage** : permet à tout nouveau développeur de reprendre rapidement le code.
- **Performance suffisante** pour le volume visé (quelques centaines d'items et prêts simultanés).
- **Connexion native via PDO** : gestion simplifiée des requêtes préparées, sécurité accrue et code plus concis.

Ce choix garantit une maintenance aisée, un temps de développement réduit et une intégration fluide avec le modèle de données relationnel défini.

### **13.3 DocumentRoot = public/ et .htaccess : sécurité et routage maîtrisés**

Le *DocumentRoot* d'*Apache* a été redéfini vers *public/*, ce qui permet de :

- ne jamais exposer le code applicatif (*src/*, *config/*) au web ;
- simplifier le routage via un seul fichier *.htaccess* ;
- concentrer le point d'entrée unique de l'application (*index.php*).

Le *.htaccess* redirige les URLs internes vers */frontend/accueil.html*.

Cette approche est standard dans les frameworks modernes et renforce la sécurité tout en gardant une architecture claire.

### **13.4 API sous public/api : séparation claire front / back**

Les *scripts d'API (PHP)* ont été regroupés sous *public/api/* (ex. *addpret.php*, *getactiveloads.php*, *cloturepret.php*).

Le frontend reste 100 % statique : il se compose d'HTML/CSS/JS, consommant les endpoints via *fetch()*.

#### Bénéfices :

- **Découplage clair** entre la présentation (frontend) et la logique métier (backend).
- **Maintenance simplifiée** : chaque endpoint gère une seule opération métier.
- **Facilité d'évolution** : les endpoints peuvent être réutilisés pour une application mobile ou une interface tierce.

Cette architecture MVC légère respecte le principe de séparation des responsabilités.

### **13.5 QR = ID : efficacité du flux de prêt / restitution**

Le *QR code* encode directement l'identifiant unique du matériel (*id\_item*).

Ainsi, le scan d'un QR lance immédiatement la requête correspondante vers l'API, sans recherche intermédiaire.

#### Résultats :

- Processus de prêt et restitution instantané ;

- Moins d'allers-retours serveur / base de données ;
- Moins d'erreurs humaines (aucune saisie manuelle).

Ce choix simplifie le workflow et optimise la vitesse d'exécution sur le terrain.

## 13.6 Archivage logique : conformité et traçabilité

L'*archivage logique* (champ *archived*) a été préféré à la suppression physique des données. Les entrées marquées comme archivées restent consultables pour les rapports et audits.

Avantages :

- **Aucune perte d'historique** : les prêts passés restent consultables.
- **Conformité RGPD** : possibilité d'anonymiser les utilisateurs sans casser les liens de données.
- **Facilité d'audit** : toutes les opérations sont retraçables.

Ce mécanisme assure une traçabilité complète tout en respectant les exigences de confidentialité et de gouvernance des données.

## 14. Guide “Première contribution” (Check-list – 15 minutes)

### 14.1 Objectif

Cette section aide tout nouveau développeur à installer, comprendre et contribuer au projet *MédiaStock* en moins de 15 minutes.

Elle couvre les étapes d'installation locale, d'exploration du code et de participation via Git.

### 14.2 Étapes d'installation rapide

#### 1. Cloner le dépôt GitHub

```
git clone https://github.com/....//MediaStock.git
cd MediaStock
```

#### 2. Créer le fichier d'environnement

```
cp .env.example .env
```

Renseigner les variables : mots de passe MySQL, nom de base, ports si nécessaire.

#### 3. Lancer l'environnement Docker

```
docker-compose up -d --build
```

Vérifier ensuite :

- <http://localhost:9080> → application web
- <http://localhost:8082> → phpMyAdmin (mode développement uniquement)

### 14.3 Découverte du code

- Parcourir le dossier *public/frontend/* : structure et logique d'interface (HTML, PHP, CSS, JS).
- Explorer *public/api/* : endpoints côté serveur consommés par le frontend.

- Lire *config/database.php* et *src/* pour comprendre la logique PDO et les interactions avec la base.

**Astuce :** suivre le flux complet d'un prêt :

*creation-pret.php* → *fetch* → *addpret.php* → *insertion en base* → *retour JSON*.

## 14.4 Contribution via Git

1. Créer une branche à partir de *main*  
*git checkout -b nom\_de\_la\_personne*
2. Faire les modifications, tester localement.
3. Commit avec un message clair selon Conventional Commits, par ex. :  
*feat(api): ajout d'un endpoint de filtrage par catégorie*
4. Créer une merge request vers votre branche ou *main* selon le workflow.
5. Après validation, la branche est fusionnée.

## 14.5 Bonnes pratiques à respecter

- Ne jamais versionner le fichier *.env* (*confidentiel*).
- Vérifier les *logs* avant tout commit : *docker compose logs web*
- Forcer le rechargement navigateur après un pull : *CTRL + F5*.
- Respecter la structure des endpoints et le style du code existant.
- Tester chaque scénario de prêt/restitution avant push.

# 15. Retour d'expérience des développeurs

## 15.1 Difficultés rencontrées

Lors de la réalisation du projet *MédiaStock*, plusieurs défis techniques et organisationnels ont été rencontrés, notamment dans la gestion de version, la coordination d'équipe et l'intégration front-back.

### Répartition des tâches et coordination

La répartition des tâches entre les membres n'a pas toujours été évidente.

Au démarrage du projet, il était parfois difficile de déterminer qui devait faire quoi, ce qui a entraîné:

- des chevauchements dans certaines parties du code,
- des retards dans la livraison de modules,
- et des différences de logique entre les composants développés.

Ces difficultés ont mis en évidence l'importance d'une planification précise et d'un suivi clair des responsabilités.

### Utilisation de Git en équipe (première collaboration sur un même dépôt)

Ce projet constituait pour la plupart d'entre nous la première utilisation collaborative de Git et GitHub, et cela a entraîné plusieurs problèmes :

- Certains membres oubliaient d'effectuer un *git pull* avant de commencer à coder, provoquant des divergences entre les branches locales et le dépôt distant.
- Après un *git push* mal synchronisé, il arrivait que du code ancien soit réintroduit ou que des fichiers récents disparaissent.
- Les *commits* irréguliers compliquaient le suivi des modifications et ralentissaient la progression : lorsqu'un membre de l'équipe était absent et n'avait pas poussé son code, une partie du projet devait parfois être refaite ou corrigée manuellement.
- Le changement de branches (*main*, ou la branche d'un autre développeur) n'était pas toujours bien maîtrisé, entraînant parfois des erreurs de fusion ou des conflits.
- Dans certains cas, Visual Studio Code ne mettait pas à jour automatiquement les fichiers après un *git pull*, ce qui nécessitait une vigilance accrue et obligeait à recharger manuellement le projet pour récupérer les dernières modifications.

Ces problèmes ont montré qu'une maîtrise rigoureuse du workflow Git est essentielle dans un projet collaboratif.

### Déroulement du développement : backend puis frontend

Nous avons commencé le projet par le développement du **backend (PHP + MySQL)** afin de construire la base de données, les modèles PDO et les endpoints API.

Le **frontend (HTML/CSS/JS)** a été développé dans un second temps, mais cette approche a entraîné plusieurs difficultés d'intégration :

- Le frontend, étant dynamique, devait s'appuyer sur des données que le backend n'exposait pas encore.  
Pour pouvoir avancer, nous avons créé de faux items et de fausses données de prêts afin de remplir dynamiquement les pages, comme :
  - la page de création de prêt,
  - la liste des matériels,
  - ou les modales d'affichage de détails.
- Par la suite, il a fallu réadapter tout le frontend une fois le backend réellement fonctionnel : ajustement des *appels fetch()*, des structures JSON et des noms de variables.
- Côté backend également, il n'était pas toujours clair quelles requêtes SQL ou endpoints seraient nécessaires pour alimenter correctement les composants dynamiques du frontend. Certaines routes ont dû être redéfinies ou complétées après coup, ce qui a occasionné des ajustements dans la logique côté serveur et dans les scripts JavaScript correspondants.

Ce décalage entre les deux phases (backend puis frontend) a été formateur : il nous a appris l'importance de concevoir en parallèle la structure API et l'interface utilisateur pour éviter les réadaptations répétées.

## 15.2 Enseignements et axes d'amélioration

Malgré ces difficultés, cette *expérience* s'est révélée *très enrichissante*.

Elle nous a permis d'apprendre à mieux collaborer, à structurer nos branches Git, et à anticiper les dépendances entre backend et frontend.

Pour les prochains projets, nous prévoyons de :

- Définir dès le début un *workflow Git* clair :
  - *git pull* systématique avant chaque modification,
  - commits fréquents et descriptifs (selon *Conventional Commits*),
  - validations collectives avant chaque *merge* dans *main*.
- Mettre en place un tableau de répartition des tâches (Trello, Notion ou GitHub Projects) pour mieux suivre les avancées et éviter les chevauchements.
- Travailler en itérations parallèles entre backend et frontend : concevoir ensemble les structures JSON, les endpoints et les formats de réponse avant de coder.
- Organiser des réunions de synchronisation rapides (*stand-up meetings*) pour coordonner les travaux de la journée.
- Créer un guide interne Git (checklist avant push/pull, création de branches, résolution de conflits, bonnes pratiques de merge).
- Se former davantage sur les commandes avancées Git (rebase, stash, merge, reset) pour mieux gérer les conflits.

## 15.3 Bilan humain et technique

Ce projet a été une véritable *leçon de travail d'équipe*.

Nous avons pris conscience que la réussite d'un développement collaboratif ne dépend pas seulement de la technique, mais surtout de la *communication, de la rigueur et de la coordination*.

Malgré les erreurs, cette première expérience nous a permis :

- d'acquérir une meilleure maîtrise de Git et de la collaboration en branches,
- de comprendre l'importance d'un développement itératif et coordonné entre front et back,
- et de renforcer notre capacité à gérer collectivement les difficultés techniques.

Nous terminons ce projet avec une expérience concrète du travail en équipe sur un environnement Docker + GitHub, et une volonté forte de nous améliorer sur les aspects collaboration, versionnement et intégration continue pour nos futurs projets.

## 15.4 Évolutions prévues et perspectives d'évolution

Dans une optique de maintenance évolutive et de reprise du projet par d'autres développeurs, plusieurs améliorations fonctionnelles et techniques ont été identifiées pour une version ultérieure de l'application **MédiaStock**.

Ces évolutions ne remettent pas en cause le socle actuel, mais visent à renforcer l'ergonomie, la sécurité et les capacités d'analyse de l'application dans un contexte de déploiement réel.



### **Améliorations fonctionnelles**

- **Recherche avancée par nom d'emprunteur**, en complément des filtres existants (catégorie, état, disponibilité), afin de faciliter l'accès rapide aux prêts et matériels associés à une personne.
- **Amélioration de la saisie de la date de retour**, avec la possibilité de définir la date de retour au même jour que la date de début du prêt, afin de mieux gérer les prêts de courte durée.
- **Ajout d'un bouton de déconnexion explicite** pour l'administrateur, complétant la déconnexion automatique par expiration de session.
- **Inventaire par scan de QR code**, permettant d'accéder directement à la fiche d'un matériel sans navigation préalable dans l'interface.
- **Notifications par e-mail** aux emprunteurs en retard, afin d'automatiser les rappels de restitution.
- **Archivage et désarchivage des emprunteurs**, sans suppression de l'historique des prêts, afin de préserver la traçabilité tout en désactivant les comptes inactifs

### **Évolutions techniques et organisationnelles**

- **Gestion multi-utilisateurs**, avec la possibilité de créer plusieurs comptes administrateurs ou profils distincts.
- **Gestion avancée des rôles et permissions** (administrateur, technicien, intervenant), afin de restreindre certaines actions selon le profil.
- **Journal d'audit / historique des actions**, retraçant les opérations sensibles (création, modification, prêt, restitution), pour renforcer la traçabilité.
- **Statistiques et tableaux de bord**, permettant d'analyser l'usage de l'application (nombre de prêts, retards, matériels les plus empruntés, répartition par catégorie).
- **Gestion du droit à l'effacement (RGPD)** : suppression des données sur demande de l'utilisateur, fonctionnalité prévue dans une version ultérieure, conformément aux obligations réglementaires.

Ces pistes d'évolution constituent une base de travail claire pour les futures itérations du projet et facilitent la prise en main du code par une nouvelle équipe de développement.

### **Fin du document — Fiche technique du projet *MédiaStock***

*(Document réalisé par l'équipe de développement – année 2025)*